

面向对象程序设计

第04章 继承与多态

赵毅力 ylzhao@swfu.edu.cn

大数据与智能工程学院

西南林业大学

面向对象程序设计

封装

继承

多态

继承

目的：模拟现实世界中的继承关系

作用1：代码重用

作用2：实现多态

声明子类

- 使用关键字**extends**定义一个类的**子类**，其格式如下：

```
class 子类名 extends 父类名 {  
    ...  
}
```

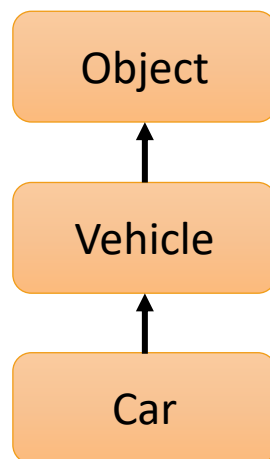
- 例如：

```
class Car extends Vehicle {  
    ...  
}
```

说明：把**Car**类定义为**Vehicle**类的子类、**Vehicle**类是**Car**类的父类。

类的继承构成树形结构

- Java中的类按继承关系形成**树形结构**。在这颗树形结构中，根节点是**Object**类，即**Object**类是所有类的祖先类。
- 除了**Object**类，每个类都**有且仅有一个父类**（单继承），一个类可以有多个或零个子类。
- 如果一个类（除了**Object**类）的声明中没有使用**extends**关键字，这个类被系统默认为是**Object**类的子类，即类声明“class A”与“**class A extends Object**”是等同的。



子类的继承性

- 一个类包含两种成员：**数据成员**和**方法成员**。子类的成员中有一部分是**子类自己声明定义**的，而另一部分是从它的父类**继承**的。
- 所谓子类继承父类的数据成员就是把继承来的数据作为自己的一个成员变量，**就好象它们是在子类中直接声明一样**，可以被子类中自己定义的任何实例方法操作。（**取决于成员的访问修饰符**）
- 所谓子类继承父类的方法成员就是把继承来的方法作为子类中的一个方法，**就好象它们是在子类中直接定义了一样**，可以被子类中自己定义的任何实例方法调用。（**取决于成员的访问修饰符**）

类成员的访问修饰符

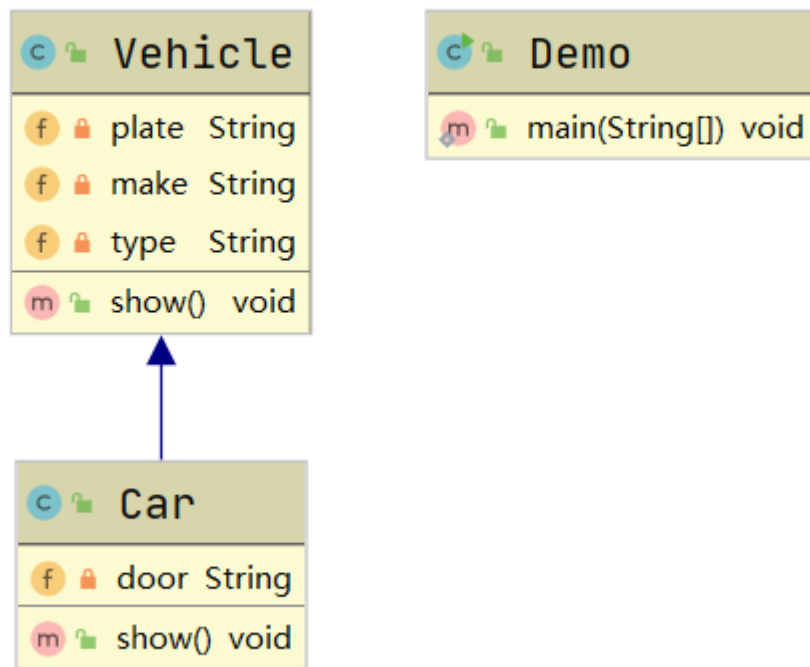
访问修饰符	含义
private	私有访问（只有类自己才能访问）
无	包访问（位于同一个包中的不同类可以互相访问）
protected	保护访问（只有类自己以及子类才能访问）
public	公有访问（没有访问限制）

类成员的访问控制

	同一个类中	同一个包中	不同包中的子类	不同包中的非子类
private	✓	×	×	×
默认	✓	✓	×	×
protected	✓	✓	✓	×
public	✓	✓	✓	✓

继承关系的UML图

- 可以直接在IntelliJ IDEA（专业版）中生成类继承关系的UML图。



子类对象的生成

- 创建子类的对象时，**子类的构造方法**总是先调用父类的某个构造方法，完成父类部分的创建；然后再调用子类自己的构造方法，完成子类部分的创建。
- 如果子类的构造方法没有明显地指明使用父类的哪个构造方法，子类就调用父类的不带参数的构造方法。
- 在子类的构造方法中可以通过**super关键字**调用父类的构造方法。

方法重写的概念

- 如果子类继承了父类的实例方法，那么子类就可以对这个方法进行重写。
- 在Java语言中，类的所有实例方法默认都是可以重写的。（除了构造方法）
 - 备注：C++和C#中类的所有实例方法默认都是不可重写的。
- 在程序设计语言中，可以被重写的方法又称为虚拟方法，重写操作被称为 **override**。
 - 备注：C++和C#使用关键字 **virtual** 声明虚拟方法。

方法重写的目的

- 代码重用
- 子类通过方法重写：当子类对象和父类对象收到同一个消息时能够有不同的响应，这在面向对象程序设计中被称为多态。
- 在面向对象程序设计中，正是通过方法重写来实现多态。

重写方法的调用

- 通过子类创建的对象，如果子类重写了父类的方法，则运行时系统调用的是子类重写的方法。
- 通过子类创建的对象，如果子类未重写父类的方法，则运行时系统调用的是子类继承的方法。
- 可以在子类重写方法中通过super关键字调用父类同名方法。

对象的向上转型

- 在类的继承树结构中，可以把一个子类对象的引用向上转型为父类。
 - `Car car = new Car("QVR23K", "TOYOTA", "CAMERY", "4");`
 - `Vehicle vehicle = car; // 向上转型`
- 多态和向上转型的限制：向上转型后，上转型引用变量不能访问子类新增的成员变量，也不能调用子类新增的方法。

多态的限制

- Shape s = new Polygon();
- s.rotate(); // 正确✓
- s.calcArea(); // 错误×

```
public class Shape {  
    public void rotate() {}  
}  
  
public class Polygon extends Shape {  
    public void rotate() {  
        // 实现Polygon类自己的旋转方法  
    }  
    public void calcArea() {  
        // 计算多边形的面积  
    }  
}
```

- 结论：通过父类的引用变量只能访问在父类中定义的方法。

多态

- 当使用向上转型时，可以使用父类引用调用子类重写过的方法，这个过程称为**多态**。
- 在面向对象程序设计语言中，多态也称为**运行时绑定**或**动态绑定**。
- **继承**和**多态**是构成面向对象软件实现的基石。

instanceof运算符

- **instanceof:** 判断一个对象是否是某个类或其子类所创建的对象。

抽象类和抽象方法

- 用关键字**abstract**修饰的类称为抽象类。

例如: **abstract class Vehicle {**

... ..

}

- 用关键字**abstract**修饰的方法称为抽象方法。

例如: **abstract double area();**

- 抽象类有如下特点:

- 抽象类里可以有抽象方法，也可以没有。
- 对于抽象方法，只允许声明，不允许实现。
- 不能使用**new**运算符创建抽象类的对象。
- 如果一个类是抽象类的子类，它必须实现父类包含的所有抽象方法。

为什么要使用抽象类？

- 抽象类可以抽象出重要的行为标准，该行为标准用抽象方法来表示。即抽象类封装了子类必需要有的行为标准。
- 抽象类声明的引用变量可以成为其子类对象引用的上转型对象，并调用子类重写的方法，即体现子类根据抽象类里的行为标准给出的具体行为。（即实现多态机制）
- 使用面向抽象的编程，有利于设计出易维护、易扩展的程序。（需要开发者通过一定的实践和学习才能掌握）

继承控制

- 如果一个类前面加上**final**修饰符，这个类就**不能被其他类继承**。

```
public final class Triangle {  
    ...  
}  
  
public class Polygon extends Triangle {  
    // 错误： 不能从final修饰的类继承  
}
```

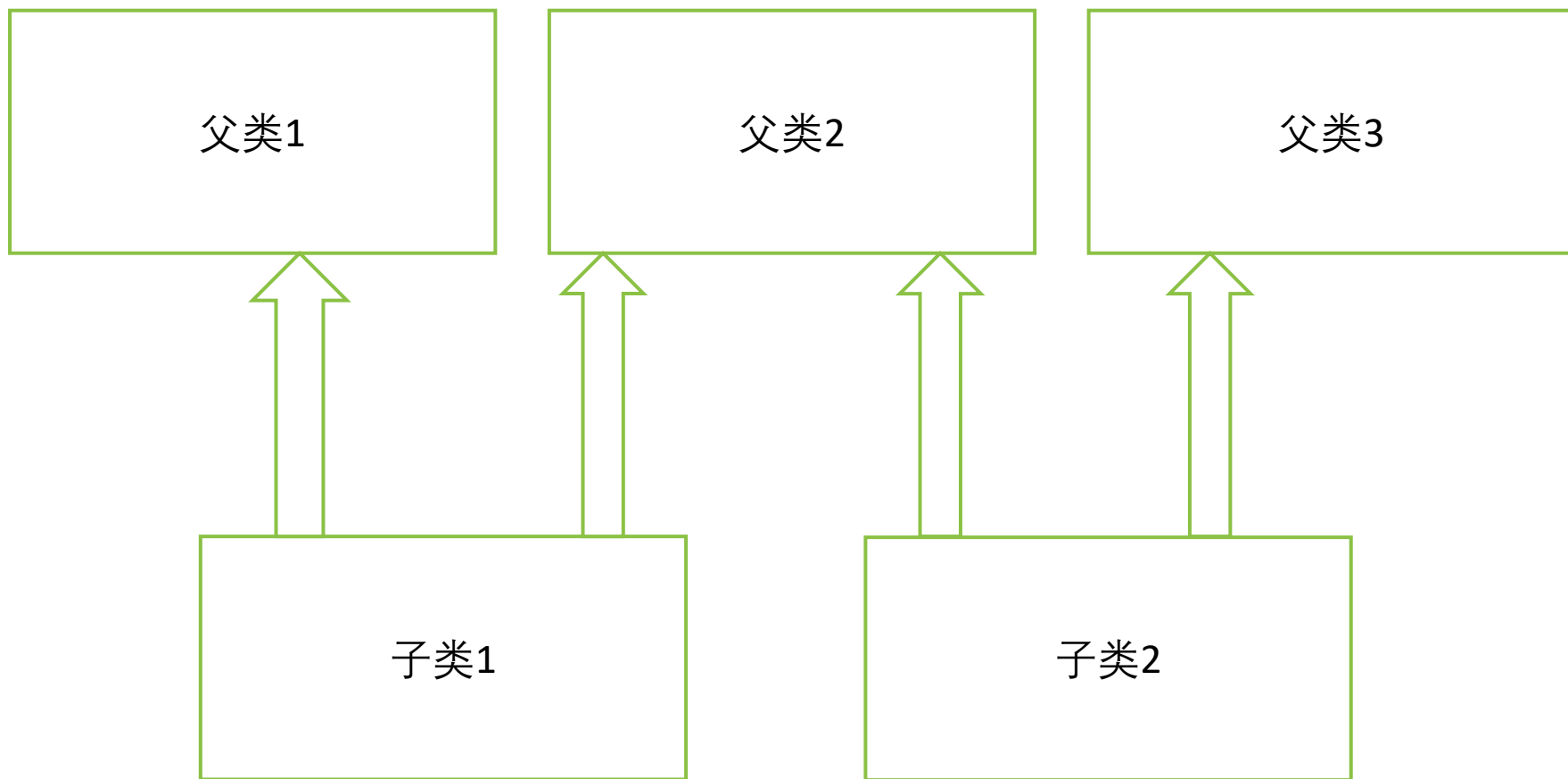
方法重写控制

- 如果一个实例方法前面加上**final**修饰符，该方法就不能被子类重写。

```
public class A {  
    public void doWork() {}  
}  
  
public class D extends A {  
    public final void doWork() {} // 取消虚函数效果  
}  
  
public class E extends D {  
    public void doWork() {} // 错误!  
}
```

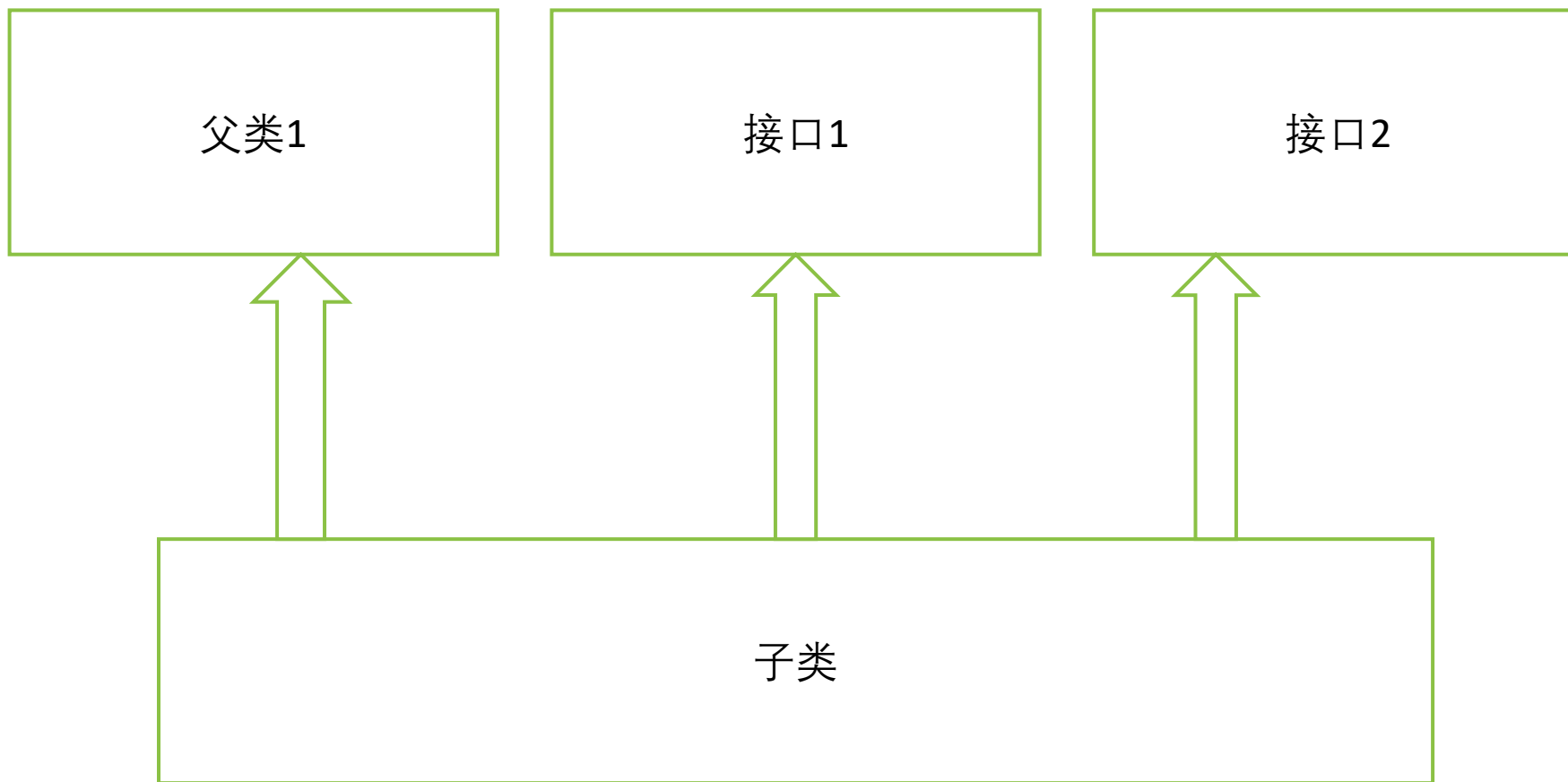
多继承

- 多继承是现实世界中存在的一种继承关系，但是Java语言不支持。



接口

- Java语言使用接口(**interface**)来模拟和实现多继承。



接口的定义

- Java语言使用关键字**interface**定义接口。

```
public interface IComputable {  
    public double compute(double a, double b); // 只有声明，没有实现。  
}
```

- 接口中的方法默认会自动加上**abstract**关键字。

接口的实现

- Java语言使用关键字**implements**实现接口。

```
public class Adder implements IComputable {  
    public double compute(double a, double b) {  
        return a + b;  
    }  
}
```

- 一个类只能**继承**一个父类，但却能**实现多个接口**！

接口与多态

- 实现一个接口的类可以当作该接口来使用。

```
public class Demo {  
    public static void main(String[] args) {  
        double a = 1.3;  
        double b = 2.7;  
        IComparable computer = new Adder(); // 多态!  
        double c = computer.compute(a, b);  
    }  
}
```

面向抽象编程

- 在设计一个程序时，可以先声明一个抽象类，通过在类中声明若干个抽象方法，表明这些方法在整个系统设计中的重要性，方法体的内容细节由它的子类去完成。
- 然后利用多态实现编程。使用多态进行程序设计的核心技术是使用方法重写和上转型对象，即将抽象类声明对象引用作为其子类的上转型对象，那么这个上转型对象就可以调用子类重写的方法。
- 所谓面向抽象编程，是指当设计某种重要的类时，不让该类面向具体的类，而是面向抽象类，即所设计类中的重要数据是抽象类声明的对象，而不是具体类声明的对象。

开-闭原则

- 所谓“开-闭原则”指的是让设计的系统应当对扩展开放，对修改关闭。
- 在设计系统时，应当首先考虑到用户需求的变化，将应对用户变化的部分设计为对扩展开放，而设计的核心部分是经过精心考虑之后确定下来的基本结构，这部分应当是对修改关闭的，即不能因为用户的需求变化而再发生变化，因为这部分不是用来应对需求变化的。
- 如果系统的设计遵守了“开-闭原则”，那么这个系统一定是易维护的，因为在系统中增加新的模块时，不必去修改系统中的核心模块。